

EC3290 – Software Requirements Engineering

Topic 5 – Requirement Evolution and Management



FACULTY OF AVIATION, SCIENCE AND TECHNOLOGY

Nilai
UNIVERSITY

Learning Objectives

This session provides a complete overview of how to manage evolving requirements in software development. You will understand the lifecycle of a requirement and how to manage it effectively as situations change.

Key Points:

- Understand the evolving nature of software requirements
- Learn to apply requirements traceability techniques
- Practice prioritization methods like MoSCoW and Kano



Explore how to manage changes using baselines

- Understand the value of reusing requirements

Introduction to Requirements Evolution

Software requirements are not static. They must adapt to changing environments, new stakeholder needs, and unforeseen challenges. Requirements evolution is a natural part of the software lifecycle.

- Requirements change due to business and technical drivers
- The need for flexibility in planning and documentation



Helps avoid outdated or irrelevant product features

What Causes Requirements to Evolve?

Several internal and external factors influence the need to update software requirements. These changes can have significant impacts on project scope and delivery.

Key Causes:

- Stakeholder feedback
- Regulatory updates
- Competitive market demands
- Technical limitations



Real-World Example - Mobile Banking App

A mobile banking app initially supports only local transactions. Later, the client requests international transfer functionality due to business expansion. Requirements evolve to include exchange rates, compliance checks, and international APIs.

Key Points:

- Reflects how growth changes user needs
- Shows how even small additions can trigger complex changes



Requirements Traceability

Traceability helps track each requirement from origin to deployment. It ensures nothing is missed and every stakeholder need is fulfilled.

Key Points:

- Supports impact analysis and auditing
- Links requirements to design, code, and tests
- Improves accountability and verification



Types of Traceability

- Forward Traceability: Requirement → Test
- Backward Traceability: Test → Requirement
- Bidirectional Traceability: Both directions



Traceability Example - Password Reset

A requirement like "User resets password via OTP" can be traced through design documents, code modules, and test cases.

Trace Links:

- Design: OTP input interface
- Code: otp_reset.py
- Test: TC003
- Deploy: Version 1.2



Tools for Traceability

- IBM DOORS
- Jira with Xray/Zephyr
- Jama Connect
- ReQtest



Requirements Prioritization

Not all requirements are created equal.

Prioritization helps teams focus on delivering the highest value first.

Benefits:

- Aligns with business goals
- Manages scope effectively
- Enhances decision-making in trade-offs



MoSCoW Technique

MoSCoW is a simple but effective prioritization method.

Categories:

- Must have: Essential
- Should have: Important
- Could have: Nice to have
- Won't have: Out of scope (for now)



Kano Model and 100-Dollar Test

Kano Categories:

- Basic Needs
- Performance Features
- Delighters

100-Dollar Test:

- Allocate imaginary budget to requirements
- Reflects perceived stakeholder value



Value vs. Complexity Matrix

Helps visualize the trade-off between value and implementation difficulty.

Quadrants:

- High value, low effort: Prioritize
- High value, high effort: Plan with caution
- Low value, low effort: Consider
- Low value, high effort: Avoid



Example - Mobile Banking App

Features

Feature	Priority
Login/Signup	Must
View Balance	Must
Fund Transfer	Should
Budget Planner	Could
Crypto Trading	Won't

Why Requirements Change

Change is inevitable in software development.
Recognizing and managing change is critical.

Reasons:

- New insights or feedback
- Legal compliance
- Feature gaps
- New technologies



Change Management Process

Steps :

- 1.Submit change request
- 2.Analyze impact
- 3.Review with stakeholders
- 4.Approve or reject
- 5.Update baseline and notify team



Requirements Baselines

A baseline freezes the requirements at a given time. All future changes must follow formal procedures.

Benefits:

- Provides a reference point
- Supports project tracking
- Controls scope creep



Configuration Management

Includes:

- Version control (e.g., Git)
- Change logs
- Release notes

Benefits:

- Reduces confusion
- Tracks evolution over time



Reusing Requirements

- Reusability leads to efficiency, especially for common modules.

Examples of Reusable Requirements:

- Login authentication
- Email notifications
- Role-based access

Advantages:

- Saves time
- Improves reliability



Recap

- Requirements are living documents. Managing them means tracking, prioritizing, controlling, and reusing.
- Traceability ensures visibility
- Prioritization helps decisions
- Change must be controlled
- Baselines keep projects stable
- Reuse builds on past work

